

SECTION D - SCENARIO-BASED INTEGRATION QUESTIONS

12 Questions · Realistic, job-style narratives integrating multiple concepts at once

These draw on genomics, clinical, finance, weather, and ML pipelines - the kind of multi-step reasoning expected in a data analyst or junior ML engineer interview, not isolated syntax recall.

Q154. [SCENARIO] (High)

```
readings = sensor_stream.get_batch()      # large 1D array
anomalies = readings[readings > threshold]
anomalies[:] = threshold                  # attempted "fix"
# dashboard still shows uncapped values in readings...
```

You are building a real-time dashboard. After running this "fix," the dashboard still shows uncapped anomaly values in `readings`. What is the root cause and the correct fix?

- a) `anomalies[:] = threshold` should have worked since arrays are always mutable
- b) Boolean-mask indexing (`readings[readings > threshold]`) returns a COPY, so modifying `anomalies` never touches `readings`; the fix is to assign directly back in one step: `readings[readings > threshold] = threshold`
- c) NumPy does not support `[:]` slice assignment
- d) The threshold value itself must be wrong

Q155. [SCENARIO] (High)

```
X = load_features()      # shape (10000, 50)
X_norm = (X - X.mean(axis=0)) / X.std(axis=0)
model.fit(X_norm)        # training silently performs badly
```

Feature column #17 happens to be constant (identical value for every row) across all 10,000 samples. What goes wrong, and what is the professional fix?

- a) Nothing goes wrong - NumPy automatically handles constant columns
- b) That column's std is 0, causing division by zero → the entire column becomes NaN, which can silently corrupt downstream model training; fix by detecting zero-std columns beforehand (e.g. `std[std==0] = 1`, or drop constant columns entirely) before normalizing
- c) NumPy raises a fatal error immediately, making the bug self-correcting
- d) Constant columns are automatically dropped by NumPy during normalization

Q156. [SCENARIO] (High)

```
sim = compute_similarity_matrix()      # shape (1000, 1000)
top10 = np.argsort(sim.flatten())[-10:]
# results include many nonsensical diagonal entries
```

The analyst wants the top 10 most similar PAIRS (excluding self-similarity on the diagonal). Two things went wrong with this code - what are they?

- a) argsort() never works correctly on flattened 2D arrays
- b) (1) The diagonal (self-similarity, always maximal) was never excluded before sorting, so it dominates the results; (2) argsort on a

FLATTENED array returns flat indices, which must be converted back to (row,col) pairs using ``np.unravel_index()`` - using the flat indices directly is meaningless c) ``flatten()`` is the bug - ``ravel()`` would have fixed it d) `argsort()` always sorts descending by default, so the ``[-10:]`` slice is backwards

Q157. [SCENARIO] *(High)*

```
returns_vectorized = prices[1:] / prices[:-1] - 1
# colleague proposes a manual for-loop "for clarity"
# ~12,500 daily price points (50-year history)
```

A teammate claims a manual for-loop will be "about the same speed" as the vectorized slice-and-divide for this ~12,500-point series. Are they right?

- a) Yes, completely correct - no meaningful difference at this size b) No - even at ~12,500 elements the vectorized approach is meaningfully faster, and more importantly far less error-prone (off-by-one "yesterday vs today" indexing errors in manual loops are a classic finance-code bug) - vectorized NumPy is the right default even at this "small" scale c) The loop would actually be faster since it avoids array overhead d) Both approaches will raise errors for negative returns

Q158. [SCENARIO] *(High)*

```
data = load_expression() # shape (20000 genes, 500 samples)
# Goal: flag genes exceeding 2 std devs from THEIR OWN gene-specific mean
```

Which single line correctly computes a boolean flag matrix of shape (20000, 500), using each gene's OWN mean and std (not a single global threshold)?

- a) ``flags = data > data.mean() + 2*data.std()`` - a single global, scalar threshold b) ``flags = data > (data.mean(axis=1, keepdims=True) + 2*data.std(axis=1, keepdims=True))`` - a per-gene threshold, broadcast across all samples c) ``flags = data > 2`` d) ``flags = data.mean(axis=0) > 2*data.std(axis=0)``

Q159. [SCENARIO] *(High)*

```
# Profiling shows 90% of runtime in this triple-nested loop:
for row in range(1080):
    for col in range(1920):
        for ch in range(3):
            if img[row,col,ch] > 200:
                img[row,col,ch] = 255
```

What is the recommended NumPy fix for this thresholding loop over a (1920, 1080, 3) image, and the expected impact?

- a) Switch to a different programming language entirely b) Replace the explicit triple-nested loop with vectorized boolean masking: ``img[img > 200] = 255``, which operates on all ~6.2 million values at once in compiled code - typically a 50-200× speedup for this kind of thresholding operation c) Reduce the image resolution to fix the performance problem d) Switch from a NumPy array to a Python list for the image data

Q160. [SCENARIO] *(High)*

```
sales = load_hourly_sales()    # shape (24, 7) - hour x day-of-week
# CFO: "Busiest hour overall, averaged across the week?
#       And which day has the single highest peak hour?"
```

Which two computations correctly answer the CFO's two questions?

- a) `sales.argmax()` answers both questions b) Busiest hour overall: `sales.mean(axis=1).argmax()` (average across days for each hour, then find the peak hour); day of highest single-hour peak: `np.unravel_index(sales.argmax(), sales.shape)[1]` (the flat-array argmax converted back to (hour,day) coordinates, taking the day component) c) `sales.sum()` answers both questions d) Both questions require exactly the same single value

Q161. [SCENARIO] (Medium)

```
print(output_v1 == output_v2)
# >>> array([ True,  True, False,  True, ...])
# "The outputs differ - there must be a bug!"
```

A QA engineer compares two versions of a simulation's output using `==` and concludes "the outputs differ," but the simulation logic is provably identical. What is the most likely explanation, and the better comparison method?

- a) The simulation genuinely has a bug - `==` is always a reliable comparison for floats b) Floating-point arithmetic accumulates tiny rounding differences (e.g. from different operation ordering), making exact `==` comparison unreliable for floats; the correct approach is `np.allclose(output_v1, output_v2)`, which allows a small numerical tolerance c) `==` does not work on NumPy arrays at all d) The arrays must have different shapes

Q162. [SCENARIO] (High)

```
# 1 million rows, manual nested-loop one-hot encoding takes 45 seconds
labels = np.array([2, 0, 1, 2, ...])    # shape (1_000_000,)
num_categories = 3
```

A colleague suggests vectorizing this one-hot encoding using broadcasting against `np.arange(num_categories)`. What is the correct expression, and why is it faster?

- a) `(labels[:, None] == np.arange(num_categories)).astype(int)` - broadcasts the (N,1) labels against the (num_categories,) range, producing an (N, num_categories) boolean matrix in one vectorized comparison, then casts to int - orders of magnitude faster than nested Python loops with per-cell assignment b) There is no way to vectorize one-hot encoding in NumPy c) `labels.reshape(-1,1) * num_categories` produces the same result d) One-hot encoding requires a separate library and cannot be done in pure NumPy

Q163. [SCENARIO] (High)

```
temps = load_weather()    # shape (365, 24, 50) - day x hour x station
# Goal: hottest temperature EVER recorded, PER STATION
```

What is the correct call to get the single hottest temperature recorded at EACH station across the entire year, regardless of day or hour?

- a) `temps.max(axis=0)` - shape (24,50): still has an hour dimension, not what is needed b) `temps.max(axis=(0,1))` - collapses BOTH the day and hour axes simultaneously, leaving shape (50,): one maximum temperature per station c) `temps.max()` - a single scalar, loses all per-station detail d) `temps.max(axis=2)` - collapses the station axis, the opposite of what is needed

Q164. [SCENARIO] *(High)*

```
expr = load_expression()    # shape (1000 patients, 20000 genes)
# Manual loop: 8 seconds for 1000 patients
results = []
for patient_row in expr:
    results.append(np.argmax(patient_row))
```

What single vectorized call replaces this entire patient-by-patient loop, and why is it faster?

- a) `expr.argmax()` - wrong, this returns ONE single flat index for the whole matrix, not one per patient b) `expr.argmax(axis=1)` - computes the column-index of the maximum value FOR EACH ROW in one vectorized pass, returning a (1000,) array; far faster than 1000 separate Python-level function calls and their associated overhead c) `expr.argmax(axis=0)` - wrong axis, this gives one result per GENE, not per patient d) `np.sort(expr)[:, -1]` - gives the maximum VALUES, not their indices

Q165. [SCENARIO] *(High)*

```
# 10 years of minute-level prices (~3.5M points)
# Manual loop: 2 minutes runtime; production budget is 5 seconds
results = []
for i in range(30, len(prices)):
    window = prices[i-30:i]
    results.append(window.std())
```

What is the recommended architectural fix to bring this rolling-volatility computation from 2 minutes down to within a 5-second budget?

- a) Just run the identical loop on a faster server b) Replace the manual sliding-window Python loop with a proper vectorized rolling-window approach (e.g. `np.lib.stride_tricks.sliding_window_view()` to create overlapping windows without copying data, then `.std(axis=-1)` across all windows in one vectorized call) - eliminates millions of individual Python-level slice and function-call operations c) Reduce the data to daily resolution instead of minute-level, sidestepping the real performance problem d) Switch to computing only the single most recent rolling value

			using NumPy in the first place.
153	B	OPTIMIZATION	BLAS-backed matrix multiplication is one of the most heavily optimized operations in all of scientific computing - routinely 100-1000× faster than a naive triple loop at this scale.
154	B	SCENARIO	This is the single most common "my filter results don't stick" bug, scaled up to a production scenario.
155	B	SCENARIO	NaN propagation through a training pipeline is a classic silent-failure scenario - the code runs without crashing, but produces garbage results.
156	B	SCENARIO	This combines two distinct, very common real-world mistakes: forgetting to mask out trivial matches, and forgetting that flat-array index results need translating back to multi-dimensional coordinates.
157	B	SCENARIO	Performance is not the only argument for vectorization - correctness and clarity matter just as much, especially for off-by-one-prone time-series indexing.
158	B	SCENARIO	axis=1 collapses across the 500 samples

			(columns) to get one mean/std PER GENE (per row); keepdims=True preserves shape (20000,1) so it broadcasts correctly back against the full (20000,500) matrix.
159	B	SCENARIO	Boolean-mask assignment handles the entire 3D array in one vectorized call, regardless of its dimensionality - no manual indexing into row/col/channel required.
160	B	SCENARIO	The first question needs an axis-collapse (average across days) followed by argmax over the remaining axis; the second needs the array's GLOBAL maximum location, requiring unravel_index to translate a flat-index argmax back into 2D coordinates.
161	B	SCENARIO	np.allclose() (and the closely related np.isclose()) exists precisely because exact floating-point equality is almost never the right comparison after any nontrivial computation.
162	A	SCENARIO	This single broadcasting comparison replaces what would otherwise be a million-iteration Python loop with per-cell

			array writes.
163	B	SCENARIO	NumPy aggregation functions accept a TUPLE of axes to collapse multiple dimensions in one call - a powerful but frequently underused feature for multi-dimensional data.
164	B	SCENARIO	Every NumPy aggregation/arg-function that accepts an axis parameter can replace an entire "loop over rows, call function on each" pattern with a single vectorized call.
165	B	SCENARIO	Verified that sliding_window_view exists and works exactly this way: it builds an overlapping-window VIEW (no data duplication) so .std(axis=-1) computes every window's statistic in one fully vectorized call - the canonical NumPy solution to "rolling computation" performance problems.